

Clasificación de variedades de arroz mediante regresión logística y Random Forest

Marco Iván Flores Villanueva
Instituto Tecnológico y de Estudios Superiores de Monterrey
Querétaro, México
A01276586@tec.mx

Abstract — Este documento presenta la implementación y evaluación de dos modelos de aprendizaje supervisado para clasificar granos de arroz de las variedades Cammeo y Osmancik a partir de sus características físicas. El dataset utilizado contiene 3810 observaciones y siete características originales, de las cuales se seleccionaron seis después de un análisis exploratorio. Primero se desarrolló un modelo de regresión logística a mano mediante la función sigmoide, binary cross-entropy y gradiente descendente. Después, se implementó un Random Forest mediante la librería scikit-learn. El dataset se dividió de manera estratificada en 60% para entrenamiento, 20% para validación y 20% para prueba, usando las mismas observaciones en ambos modelos. Como métrica principal se usó Macro-F1, complementada con accuracy, precision, recall, F1-score por clase y matrices de confusión. La regresión logística obtuvo un Macro-F1 de 0.9250 en entrenamiento, 0.9342 en validación y 0.9254 en prueba, mostrando un comportamiento consistente entre los tres conjuntos, mientras que el Random Forest con parámetros base obtuvo un Macro-F1 de 1.0000 en entrenamiento y 0.9245 en validación, evidenciando un overfitting. Para reducir esta diferencia se realizó un ajuste de hiperparámetros basado en el comportamiento observado en entrenamiento y validación. Después de ocho modificaciones, el modelo final obtuvo un Macro-F1 de 0.9267 en entrenamiento, 0.9341 en validación y 0.9241 en prueba, con una brecha de generalización de -0.0074. Estos resultados permitieron comparar la capacidad de generalización, el sesgo, la varianza y el nivel de ajuste de ambos algoritmos para el mismo dataset.

Keywords — *rice classification, logistic regression, Random Forest, hyperparameter tuning, bias, variance, overfitting, Macro-F1, cross-entropy, gradient descent*

I. INTRODUCCIÓN

Los algoritmos de machine learning permiten construir modelos capaces de aprender relaciones entre variables de entrada y una variable objetivo a partir de observaciones utilizadas durante el entrenamiento. En ese ámbito, los problemas de clasificación buscan asignar cada observación a una categoría determinada, y cuando existen únicamente dos clases, este problema puede abordarse mediante diferentes técnicas supervisadas, entre ellas la regresión logística y Random Forest.

Para este trabajo se usó el dataset Rice - Cammeo and Osmancik, compuesto por 3810 observaciones correspondientes a granos de arroz de las variedades Cammeo y Osmancik. A partir del procesamiento de imágenes de los granos se obtuvieron siete características físicas: Area, Perimeter, Major Axis Length, Minor Axis Length, Eccentricity, Convex Area y Extent. Estas variables describen diferentes propiedades relacionadas con el tamaño y la forma de los granos y permiten estudiar si existe suficiente información para distinguir automáticamente entre ambas variedades.

El objetivo de este trabajo es implementar, evaluar y comparar dos modelos de aprendizaje supervisado para

clasificar los granos de arroz a partir de sus características físicas. El primer modelo consiste en una regresión logística implementada manualmente con NumPy, mientras que el segundo corresponde a un Random Forest implementado mediante scikit-learn. Ambos modelos se entrenan y evalúan usando los mismos conjuntos de entrenamiento, validación y prueba, permitiendo realizar una comparación consistente de su desempeño.

Además de evaluar su capacidad de clasificación, se analiza el sesgo, la varianza y el nivel de ajuste de cada modelo. Para esta evaluación se utiliza Macro-F1 como métrica principal, complementada con accuracy, precision, recall, F1-score por clase y matrices de confusión. En el caso de Random Forest, el modelo base es posteriormente sometido a un ajuste de hiperparámetros, donde cada modificación se selecciona a partir del comportamiento observado en entrenamiento y validación hasta lograr reducir el overfitting sin deteriorar la capacidad de generalización.

II. DATASET Y EDA

A. Descripción del dataset

El dataset Rice (Cammeo and Osmancik), publicado en el UCI Machine Learning Repository en 2019, contiene 3810 observaciones correspondientes a granos de arroz de las variedades Cammeo y Osmancik. Los datos fueron obtenidos mediante el procesamiento de imágenes de los granos, a partir de las cuales se extrajeron siete características físicas: *Area*, *Perimeter*, *Major Axis Length*, *Minor Axis Length*, *Eccentricity*, *Convex Area* y *Extent*. Estas variables describen propiedades relacionadas con el tamaño y la forma de cada grano, mientras que la variable objetivo *Class* identifica su variedad como Cammeo u Osmancik. Por lo tanto, el dataset representa un problema de clasificación binaria en el cual las características físicas se utilizan como variables predictoras para determinar la variedad correspondiente.

Variable Name	Role	Type	Description	Units
Area	Feature	Integer	Returns the number of pixels within the boundaries of the rice grain	px
Perimeter	Feature	Continuous	Calculates the circumference by calculating the distance between pixels around the boundaries of the rice grain	px
Major_Axis_Length	Feature	Continuous	The longest line that can be drawn on the rice grain, i.e. the main axis distance, gives	
Minor_Axis_Length	Feature	Continuous	The shortest line that can be drawn on the rice grain, i.e. the small axis distance, gives	
Eccentricity	Feature	Continuous	It measures how round the ellipse, which has the same moments as the rice grain, is	
Convex_Area	Feature	Integer	Returns the pixel count of the smallest convex shell of the region formed by the rice grain	
Extent	Feature	Continuous	Returns the ratio of the region formed by the rice grain to the bounding box	
Class	Target	Binary	Cammeo and Osmancik	

B. Revisión de los datos y limpieza

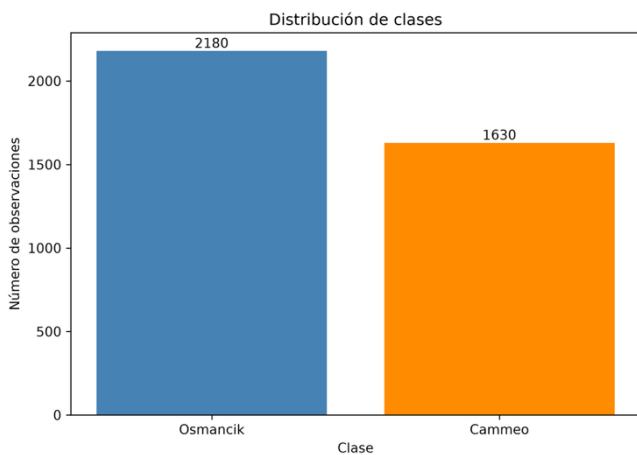
Antes de construir los modelos se realizó una revisión de la estructura y calidad de los datos. El dataset contiene 3810 observaciones y ocho columnas, con siete variables predictoras numéricas y la variable categórica *Class*. *Area* y *Convex Area* son valores enteros, mientras que las otras cinco características son variables de punto flotante.

De igual forma se buscaron valores faltantes y registros duplicados, obteniendo resultados favorables no encontrar inconsistencias en ninguna de las ocho columnas, por lo que no fue necesario realizar imputación ni eliminar registros antes del análisis.

C. Distribución de clases

La variable objetivo se encuentra distribuida en 2180 observaciones de Osmancik y 1630 de Cammeo, correspondientes aproximadamente al 57.22% y 42.78% del dataset respectivamente. Por lo tanto, sí existe un desbalance entre las clases, aunque ambas cuentan con una cantidad considerable de observaciones.

Esta diferencia se tomó en consideración durante la preparación y evaluación de los modelos. La separación de los datos se realizó de manera estratificada para conservar de manera aproximada esta misma proporción en los conjuntos de entrenamiento, validación y prueba.

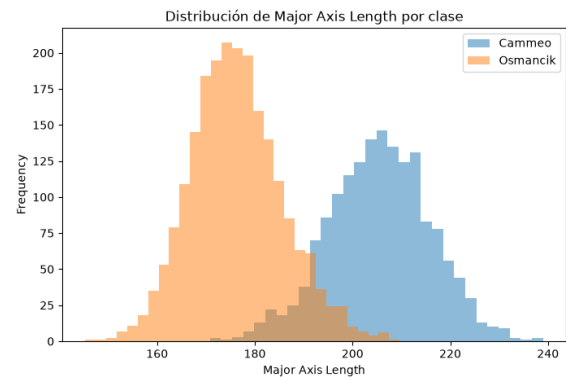


D. Análisis de las características físicas

Para determinar si las características físicas presentaban diferencias entre las dos variedades, se calcularon medidas descriptivas para cada clase y se compararon las distribuciones de las variables. Se observaron diferencias particularmente visibles en las características relacionadas con el tamaño del grano. Por ejemplo, el área promedio fue de aproximadamente 14163 para Cammeo y 11550 para Osmancik. De igual forma, Cammeo presentó mayores valores promedio de *Perimeter*, *Major Axis Length* y *Convex Area*.

También se identificaron diferencias en características relacionadas con la forma. *Eccentricity* presentó un promedio mayor para Cammeo, mientras que *Extent* mostró un promedio ligeramente mayor para Osmancik. Estas diferencias indican que las características contienen información útil para distinguir entre las dos variedades, aunque las distribuciones no se encuentran completamente

separadas.

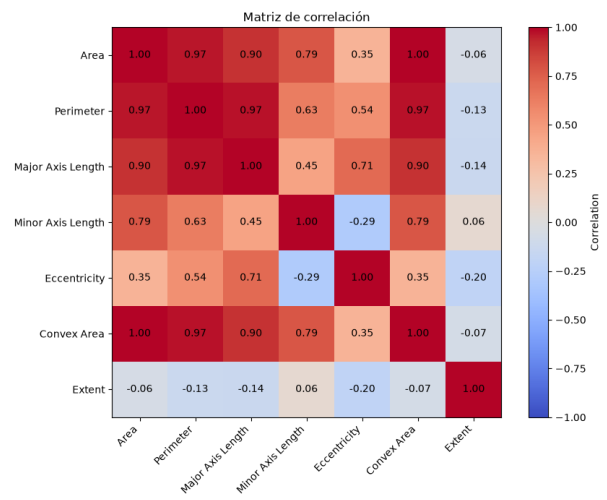


En la figura anterior se muestra un ejemplo de estas diferencias mediante la distribución de *Major Axis Length*. La distribución correspondiente a Cammeo se encuentra desplazada generalmente hacia valores mayores, mientras que Osmancik se concentra en longitudes menores. Sin embargo, también existe una región de superposición entre ambas clases. Esto indica que esta característica contiene información discriminante, pero por sí sola no permite separar correctamente todas las observaciones, justificando el uso conjunto de múltiples variables en los modelos de clasificación a continuación.

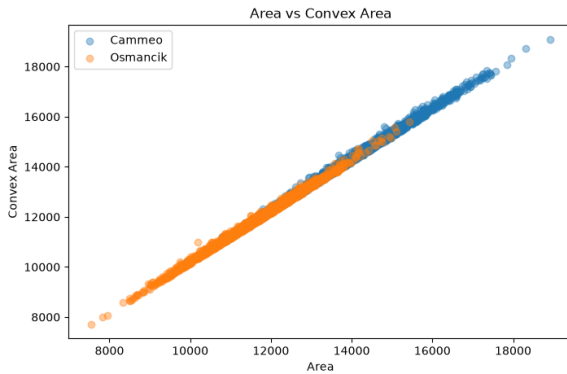
E. Relaciones entre las variables

Además de estudiar las características de manera individual, se calculó una matriz de correlación de Pearson entre las siete variables numéricas para identificar relaciones lineales y posibles fuentes de redundancia. Se observaron correlaciones positivas elevadas entre varias características relacionadas con el tamaño del grano, con *Area* y *Convex Area* presentando la más alta, de aproximadamente 0.999. También se observaron otras correlaciones altas de aproximadamente 0.972 entre *Perimeter* y *Major Axis Length* y de 0.966 entre *Area* y *Perimeter*.

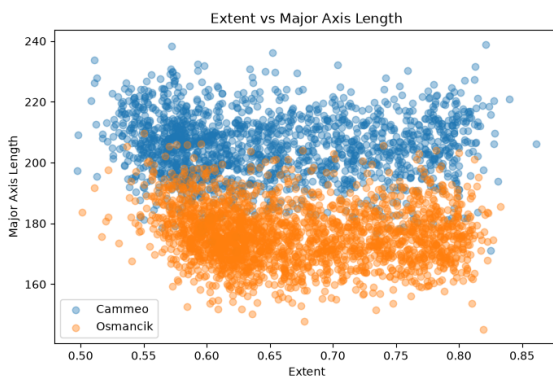
Por otro lado, *Extent* presentó correlaciones relativamente bajas con las demás características. Esto puede explicarse porque a diferencia de otras variables que miden directamente dimensiones o tamaño del grano, *Extent* representa la proporción entre el área ocupada por el grano y el área de su bounding box. Por lo tanto, esta variable captura un aspecto diferente de la geometría del grano y puede aportar información complementaria.



Podemos observar en la siguiente figura que la relación entre *Area* y *Convex Area* es lineal, pues su correlación es de 0.999, y conforme aumenta el área del grano, su área convexa también lo hace en la misma proporción. La gráfica muestra una separación parcial entre las variedades, pues Osmancik se concentra mayormente en valores menores y Cammeo en valores mayores.



En contraste, la relación entre *Extent* y *Major Axis Length*, mostrada en la siguiente figura, presenta un comportamiento distinto. Para valores similares de *Extent* pueden encontrarse observaciones con longitudes del eje mayor considerablemente diferentes y no se aprecia una relación lineal fuerte entre ambas variables. Este comportamiento refuerza la idea de que *Extent* captura información geométrica diferente a la proporcionada por las variables directamente relacionadas con el tamaño.



F. Selección de variables

Los resultados del análisis de correlación se usaron para realizar una selección de variables antes del entrenamiento de los modelos. Particularmente, la correlación de aproximadamente 0.999 entre *Area* y *Convex Area* indicó que ambas variables contenían información altamente redundante. Mantener ambas características podría aportar poca información adicional, y en la regresión logística, incrementar los riesgos asociados con multicolinealidad y dificultar la interpretación de los coeficientes individuales.

Debido a esto, se decidió retirar *Convex Area* y conservar *Area* como variable predictora. Después de esta selección, los dos modelos utilizaron las mismas seis características: *Area*, *Perimeter*, *Major Axis Length*, *Minor Axis Length*, *Eccentricity* y *Extent*.

Al retirar esta variable, se logra reducir la redundancia de los datos y mantener un conjunto de entrada consistente para comparar la regresión logística y Random Forest bajo las

mismas condiciones. Sin embargo, la matriz muestra que aún hay correlaciones altas entre algunas de las variables restantes, como entre *Perimeter* y *Major Axis Length* o *Area* y *Perimeter*, algo a tener en cuenta al interpretar los coeficientes y sus niveles de significancia estadística en la regresión logística más adelante.

III. PREPARACIÓN DE DATOS Y METODOLOGÍA DE EVALUACIÓN

A. Codificación de la variable objetivo

Después del análisis exploratorio y la selección de características, fue necesario transformar la variable objetivo *Class* a una representación numérica compatible con los modelos de clasificación, por lo que se asignó el valor 0 a la variedad Osmancik y el valor 1 a Cammeo.

B. Separación en entrenamiento, validación y prueba

Como el dataset original se encontraba ordenado de acuerdo con la variedad de arroz, y las clases estaban desbalanceadas, una separación regular habría generado subconjuntos con distribuciones diferentes, así que para evitar este problema se realizó una división estratificada y aleatoria utilizando una seed, de manera que el procedimiento fuera reproducible en cada ejecución del script.

Se asignó el 60% de las observaciones al conjunto de entrenamiento, el 20% al conjunto de validación y el 20% restante al conjunto de prueba, haciendo que primero quedara formado por 2286 observaciones, mientras que los últimos dos con 762 observaciones cada uno. Gracias a la estratificación se conservó la distribución original de las clases, con el conjunto de entrenamiento conteniendo 1308 observaciones de Osmancik y 978 de Cammeo, mientras que tanto validation como test recibieron 436 observaciones de Osmancik y 326 de Cammeo.

Adicionalmente, para garantizar una comparación consistente, se usaron exactamente las mismas observaciones en los conjuntos de entrenamiento, validación y prueba para ambos modelos. De esta manera, tanto la regresión logística como Random Forest fueron entrenados y evaluados sobre los mismos datos en cada etapa, evitando que las diferencias en su desempeño fueran consecuencia de divisiones distintas del dataset.

Conjunto	Total	Osmancik	Cammeo	Proporción
Entrenamiento	2286	1308	978	60%
Validación	762	436	326	20%
Prueba	762	436	326	20%

Dentro de la metodología, cada conjunto desempeñó una función diferente. Training se usó para estimar los parámetros de los modelos, validation se usó para analizar la capacidad de generalización (en el caso de Random Forest, para tomar decisiones durante el ajuste de hiperparámetros), y test se reservó para la evaluación final, siendo esa la razón por la cual no participó en el ajuste de hiperparámetros del Random Forest.

C. Estandarización de las características

Debido a que las características presentan magnitudes considerablemente diferentes, para la regresión logística se

aplicó una estandarización mediante z-score. Esta transformación centra cada característica alrededor de cero y expresa sus valores en unidades de desviación estándar, lo cual facilita el entrenamiento con gradiente descendente y evita que las diferencias de escala entre las variables afecten las actualizaciones de los parámetros.

$$z = \frac{x - \mu}{\sigma}$$

μ = Mean
 σ = Standard Deviation

En la fórmula de z-score, x representa el valor original de una característica, μ su media y σ su desviación estándar. Para evitar data leakage, la media y la desviación estándar fueron calculadas exclusivamente a partir del conjunto de entrenamiento. Posteriormente, estas mismas estadísticas se utilizaron para transformar los conjuntos de entrenamiento, validación y prueba.

Por otro lado, Random Forest no requirió esta estandarización, ya que la construcción de los árboles se basa en divisiones mediante umbrales y no depende de que las variables se encuentren expresadas en escalas semejantes.

D. Uso de los conjuntos en el desarrollo

La separación entre entrenamiento, validación y prueba se mantuvo durante todo el desarrollo. En la regresión logística, los gradientes y la actualización de los parámetros fueron calculados únicamente con el conjunto de entrenamiento. Aunque sí se registró la binary cross-entropy de validation durante cada epoch para observar el comportamiento de generalización, estas observaciones no participaron en el cálculo de los gradientes ni en la actualización de los pesos.

En Random Forest, training se usó para construir los árboles y validation para analizar los efectos de cada modificación de hiperparámetros; el conjunto de prueba permaneció fuera de este proceso y sólo se consideró una vez seleccionada la configuración final.

E. Métricas de evaluación

Para evaluar ambos modelos de manera consistente se seleccionó Macro-F1 como métrica principal. Aunque las dos clases cuentan con una cantidad considerable de observaciones, Osmancik representa aproximadamente el 57.22% del dataset y Cammeo el 42.78%, así que para evaluar ambas con la misma importancia y sin importar su desbalanceo, Macro-F1 resulta ser una opción pertinente al no ser sensible a la distribución de las mismas, además de que requiere que el modelo presente un desempeño equilibrado en ambas variedades y evita que el resultado global sea dominado por la clase con mayor cantidad de observaciones

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

F1 se calcula a partir de las métricas de *precision* y *recall*, y corresponde a su media armónica. Para obtener el Macro-F1, simplemente se necesita calcular el F1-score de Osmancik y Cammeo por separado y luego sacar su promedio.

Como métricas complementarias se usaron accuracy para medir la proporción total de clasificaciones correctas, además de precision, recall y el F1-score individual de cada clase. También se usaron matrices de confusión para analizar directamente los falsos positivos y falsos negativos producidos por cada modelo.

Por último, en la regresión logística se usó binary cross-entropy como función de costo y como medida adicional para estudiar el comportamiento del modelo durante el entrenamiento. En las gráficas generadas, las curvas de pérdida de training y validation permitieron observar su convergencia y detectar posibles señales de underfitting u overfitting.

F. Criterios para diagnosticar sesgo, varianza y ajuste

Además del desempeño de cada modelo, se compararon los resultados obtenidos en entrenamiento y validación para analizar su sesgo, varianza y nivel de ajuste. Para mantener consistencia, estas comparaciones se realizaron principalmente mediante Macro-F1.

Un desempeño bajo tanto en training como en validation se consideró indicativo de posible underfitting y un mayor nivel de sesgo. Por otro lado, un desempeño bueno en training, pero bajo en validation, se interpretó como una señal de alta varianza y posible overfitting. Cuando ambos resultados fueron altos y se mantuvieron cercanos entre sí, se consideró que el modelo presentaba un nivel de ajuste adecuado.

Para cuantificar esta diferencia se calculó una brecha de generalización definida de la siguiente forma en las dos implementaciones:

Regresión logística:

```
train_val_gap = train_metrics["Macro-F1"] - val_metrics["Macro-F1"]
```

Random Forest:

```
base_f1_gap = base_train_f1 - base_val_f1
```

```
adjusted_f1_gap = adjusted_train_f1 - adjusted_val_f1
```

Una brecha positiva elevada indica que el modelo presenta un desempeño considerablemente mejor sobre los datos utilizados durante el entrenamiento que sobre observaciones no vistas. Sin embargo, se reitera que esta brecha no fue considerada como un indicador definitivo de buen modelo, pues también se tomó en cuenta el Macro-F1, las métricas por clase, las matrices de confusión, y para la regresión logística, sus curvas de pérdida.

IV. IMPLEMENTACIÓN Y RESULTADOS DE REGRESIÓN LOGÍSTICA

A. Modelo de regresión logística

El primer modelo fue una regresión logística implementada manualmente utilizando la librería numpy. Después del preprocesamiento descrito en la sección anterior, el modelo recibió como entrada las seis características estandarizadas y produjo como salida la probabilidad de que cada observación perteneciera a la clase positiva, correspondiente a Cammeo.

El modelo calcula inicialmente una combinación lineal de las características mediante un vector de pesos w y un intercepto b :

$$z = w^T x + b$$

Debido a que esta combinación puede tomar cualquier valor real, se utilizó la función sigmoide para transformar z en una probabilidad entre cero y uno:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Como Osmancik fue codificada como 0 y Cammeo como 1, la salida de la función sigmoide se interpreta como la probabilidad de que el grano pertenezca a la variedad Cammeo. Para obtener una clase final, se estableció un umbral de 0.5 donde probabilidades mayores o iguales a ese valor se clasificaron como Cammeo y las demás como Osmancik.

B. Función de costo y optimización

Para entrenar el modelo, todos los pesos y el intercepto fueron inicializados en cero, y debido a que una regresión logística produce probabilidades, se utilizó binary cross-entropy como función de costo para medir la diferencia entre las probabilidades predichas y las clases reales.

$$L_{BCE} = -\frac{1}{n} \sum_{i=1}^n (Y_i \cdot \log \hat{Y}_i + (1 - Y_i) \cdot \log (1 - \hat{Y}_i))$$

En esta fórmula n representa el número de observaciones, y_i la clase real y $\hat{y}_i$ la probabilidad estimada por el modelo. En la implementación, las probabilidades fueron limitadas a valores ligeramente superiores a cero e inferiores a uno para evitar operaciones indefinidas producidas por $\log(0)$.

Por otro lado, la optimización se realizó mediante gradiente descendente por batch. En cada epoch se calcularon las probabilidades para todas las observaciones de entrenamiento, el valor de binary cross-entropy y los gradientes de la función de costo con respecto a los pesos y al intercepto, y posteriormente, los parámetros se actualizaron en dirección contraria al gradiente con el objetivo de reducir progresivamente la función de costo.

C. Configuración del entrenamiento

El modelo de regresión logística fue entrenado mediante gradiente descendente por batch durante 3000 epochs, utilizando un learning rate de 0.03. Los seis pesos asociados a las variables predictoras y el intercepto fueron inicializados en cero antes de comenzar el proceso de optimización.

D. Convergencia durante el entrenamiento

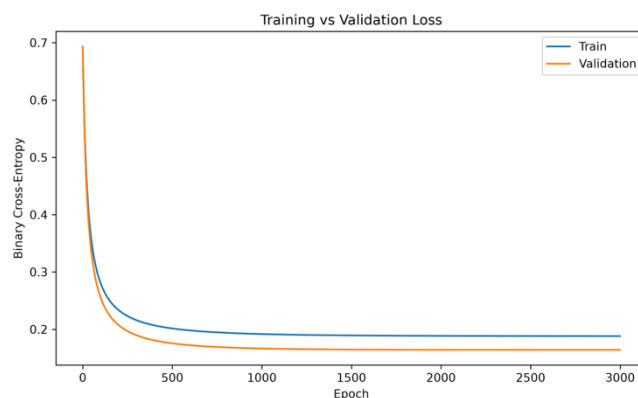
Al inicio del entrenamiento tanto la pérdida de training como la de validation fueron aproximadamente 0.69. Este valor corresponde al comportamiento inicial del modelo con todos los pesos y el intercepto en cero, lo que genera probabilidades cercanas a 0.5.

Conforme avanzó el gradiente descendente, ambas pérdidas disminuyeron rápidamente durante las primeras iteraciones y posteriormente comenzaron a estabilizarse. Alrededor de la epoch 2900, la binary cross-entropy de entrenamiento había disminuido hasta aproximadamente 0.18,

mientras que la correspondiente a validation se encontraba alrededor de 0.16.

La pérdida de validation permaneció siempre ligeramente por debajo de la pérdida de training durante el proceso. Esta diferencia puede deberse a la composición particular de los subconjuntos estratificados y no representa una señal de overfitting. Lo importante es que ambas curvas siguieron una tendencia descendente semejante y no se observó una divergencia donde la pérdida de entrenamiento continuara disminuyendo mientras la de validation aumentara.

Finalmente, en las últimas epochs, ambas curvas se aproximaron a una región de estabilización, indicando que en ese punto incrementar el número de iteraciones probablemente produciría cambios pequeños en comparación con las primeras etapas del entrenamiento.



E. Resultados en entrenamiento y validación

Después del entrenamiento, el modelo fue evaluado primero mediante los conjuntos de training y validation, y se obtuvieron los siguientes resultados:

Conjunto	Macro-F1	Accuracy
Training	0.9250	0.9265
Validation	0.9342	0.9357

Generalization gap: -0.0092

Clase	Validation precision	Validation recall	Validation F1-score
Osmancik	0.9388	0.9495	0.9441
Cammeo	0.9315	0.9172	0.9243

En validation, ambas clases obtuvieron un F1-score superior a 0.92. Osmancik presentó un F1-score ligeramente mayor que Cammeo, posiblemente por su recall más alto, lo cual indica que el modelo identificó una mayor proporción de las observaciones reales de Osmancik. De cualquier modo, la diferencia entre los F1-scores de ambas clases no fue mucha.

Validation confusion matrix	
414	22
27	299

La matriz de confusión de validation muestra que, en total, el modelo cometió 49 errores sobre las 762 observaciones del conjunto.

F. Coeficientes y significancia estadística

Además de evaluar su capacidad predictiva, se analizaron los coeficientes obtenidos por la regresión logística, y como las variables fueron previamente estandarizadas mediante z-score, los coeficientes pueden compararse utilizando una escala común. Un coeficiente positivo indica que manteniendo constantes las demás variables, un aumento de una desviación estándar en la característica correspondiente incrementa la probabilidad estimada de pertenecer a la clase Cammeo.

También se calcularon los errores estándar y los p-values asociados a cada coeficiente con el propósito de analizar su significancia estadística. Se utilizó un nivel de significancia de 0.05 para determinar si cada coeficiente presentaba evidencia estadística de ser distinto de cero.

	Variable	Coefficient	Std. Error	p-value
	Intercept	-0.746907	0.153117	0.000001
	Area	1.062857	2.561434	0.678181
	Perimeter	1.255660	1.023736	0.219993
Major Axis Length		1.395792	2.377638	0.557170
Minor Axis Length		0.166831	1.934717	0.931284
Eccentricity		1.291968	1.387509	0.351780
Extent		0.001282	0.089620	0.988588

Entre las variables predictoras, *Major Axis Length* presentó el coeficiente de mayor magnitud, seguido por *Eccentricity*, *Perimeter* y *Area*.

Todos los coeficientes fueron positivos; sin embargo, ninguna de las seis características presentó individualmente un p-value inferior a 0.05, aunque esto no implica que carezcan de utilidad para la clasificación.

Se sabe que el modelo obtuvo un Macro-F1 superior a 0.92 tanto en entrenamiento como en validación, por lo que el conjunto de variables sí contiene información predictiva relevante. No obstante, el análisis exploratorio mostró correlaciones elevadas entre varias características restantes.

Esta multicolinealidad implica que diferentes variables pueden compartir parte de la misma información predictiva, dificultando aislar el efecto individual de cada coeficiente y aumentando su incertidumbre. Es por eso que los coeficientes y sus p-values deben interpretarse juntamente con el desempeño predictivo del modelo y con las relaciones identificadas durante el EDA.

G. Diagnóstico de sesgo, varianza y nivel de ajuste

Considerando conjuntamente el desempeño de clasificación, la brecha de generalización y las curvas de loss, se diagnostica que la regresión logística presenta un nivel de sesgo bajo y una varianza baja, con un nivel de ajuste adecuado. No existen señales graves de underfitting debido al buen desempeño alcanzado, ni de overfitting, ya que el rendimiento se mantiene al pasar de training a validation.

V. IMPLEMENTACIÓN Y DIAGNÓSTICO DE RANDOM FOREST BASE

A. Configuración del modelo

Para la segunda implementación con framework se seleccionó Random Forest de scikit-learn, un algoritmo supervisado que consigue resolver problemas de clasificación mediante la combinación de varios árboles de decisión. Cada árbol se entrena a partir de una muestra aleatoria de las observaciones y considera sólo un subconjunto de las características al construir sus divisiones. La predicción final

se obtiene observando la clase seleccionada por la mayoría de los árboles.

Como cada árbol puede capturar relaciones no lineales entre las variables de entrada y la variable objetivo, Random Forest se convierte en una opción muy interesante para este dataset de arroz. Además, a diferencia de la regresión logística entrenada con gradiente descendente, Random Forest no necesita que las características se estandaricen, pues las divisiones de los árboles no dependen de la magnitud relativa de las variables.

El modelo base se entrenó usando los hiperparámetros por defecto de scikit-learn, modificando únicamente *random_state* para garantizar reproducibilidad. Esta primera configuración se tomó como referencia antes de realizar cualquier ajuste de complejidad.

B. Resultados del modelo base

El Random Forest base alcanzó resultados interesantes en su ejecución:

Conjunto	Macro-F1	Accuracy
Training	1.0	1.0
Validation	0.9245	0.9265

Generalization gap: 0.0755

Clase	Validation precision	Validation recall	Validation F1-score
Osmancik	0.9222	0.9518	0.9368
Cammeo	0.9327	0.8926	0.9122

El desempeño perfecto en training, seguido por una disminución en validation, indica que una parte de los patrones aprendidos no se generaliza a observaciones no utilizadas durante el entrenamiento.

C. Diagnóstico de sesgo, varianza y ajuste

El Macro-F1 perfecto de training indica que el modelo base tiene suficiente capacidad para representar los datos de entrenamiento y no muestra señales de underfitting. Sin embargo, la gran diferencia con el 0.9245 de validation y la brecha de 0.0755 indican una varianza elevada.

En consecuencia, el Random Forest base presenta sesgo bajo, varianza alta y overfitting. Es a partir de este diagnóstico que se decidió restringir progresivamente la complejidad de los árboles mediante un ajuste de hiperparámetros.

VI. AJUSTE DE HIPERPARÁMETROS

A. Estrategia de ajuste

Debido al overfitting observado en el Random Forest base, se decidió realizar un ajuste manual de los hiperparámetros relacionados con la complejidad de los árboles. Durante este proceso se utilizaron únicamente training y validation, y se consideraron tres aspectos principales: mantener un Macro-F1 alto en validation, reducir la brecha de generalización y evitar una caída excesiva en training que pudiera indicar underfitting.

B. Proceso de ajuste

Ajuste	Modificación principal	Train Macro-F1	Validation Macro-F1	Gap
Base	Parámetros default	1.0	0.9245	0.0755
1	max_depth=10	0.9884	0.9244	0.0639
2	max_depth=5	0.9378	0.9275	0.0104
3	max_depth=3	0.9295	0.9288	0.0007
4	max_depth=2	0.9267	0.9341	-0.0074
5	max_depth=1	0.9109	0.9197	-0.0089
6	max_depth=2, class_weight="balanced"	0.9277	0.9303	-0.0025
7	max_depth=2, max_features=0.5	0.9264	0.9303	-0.0039
8	max_depth=2, n_estimators=500	0.9271	0.9328	-0.0057

Los primeros ajustes se concentraron en max_depth, ya que el modelo base presentaba overfitting. Reducir la profundidad de 10 a 5 y posteriormente a 3 disminuyó progresivamente la brecha entre training y validation sin perjudicar el Macro-F1 de validation. Con max_depth=2 se obtuvo el mejor resultado de esta etapa, con un Macro-F1 de 0.9341 en validation y una brecha de -0.0074. Sin embargo, al reducir la profundidad a 1, el desempeño disminuyó tanto en training como en validation, indicando que los árboles habían perdido demasiada capacidad y comenzaban a presentar underfitting. Por esta razón, max_depth=2 se conservó como la profundidad seleccionada.

A partir de esta configuración se evaluaron otros hiperparámetros. El uso de class_weight="balanced" redujo el Macro-F1 de validation a 0.9303, por lo que no se obtuvo evidencia de una mejora global al ponderar las clases y se conservó class_weight=None. Igualmente, aumentar max_features a 0.5 tampoco mejoró el resultado, por lo que se conservaron class_weight=None y max_features="sqrt".

Finalmente, aumentar n_estimators de 100 a 500 mantuvo un desempeño similar, pero el Macro-F1 de validation disminuyó ligeramente a 0.9328. Dado que un mayor número de árboles tampoco aportó una mejora, se conservaron 100 estimadores.

En conjunto, los experimentos mostraron que el principal factor de mejora fue limitar la profundidad de los árboles, mientras que los demás ajustes no superaron el desempeño obtenido con max_depth=2.

C. Efecto de los hiperparámetros evaluados

Los experimentos mostraron que max_depth fue el hiperparámetro con mayor efecto sobre el comportamiento del modelo. Reducir la profundidad limitó la capacidad de los árboles para generar reglas demasiado específicas y disminuyó progresivamente el overfitting del modelo base. Sin embargo, max_depth=1 restringió excesivamente su capacidad, provocando una caída simultánea en training y validation.

Una vez seleccionada una profundidad de 2, se evaluaron class_weight, max_features y n_estimators. Ninguno de estos cambios superó el Macro-F1 de validation obtenido con sus valores por defecto, por lo que no fueron incorporados al modelo final. Esto indica que para este problema, la mejora dependió principalmente de controlar la complejidad individual de los árboles y no de aumentar el tamaño del bosque o modificar la selección de características por división.

D. Configuración seleccionada

Después de ocho ajustes adicionales al modelo base, se seleccionaron los siguientes hiperparámetros:

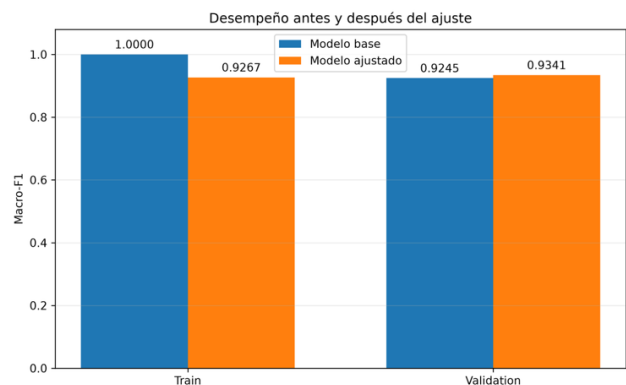
Hiperparámetro	Valor
n_estimators	100
max_depth	2
min_samples_split	2
min_samples_leaf	1
max_features	"sqrt"
class_weight	None
random_state	67

Al final, se seleccionó max_depth=2 junto con los valores por defecto de los demás hiperparámetros evaluados. Esta configuración obtuvo el mayor Macro-F1 de validation entre las corridas realizadas, con 0.9341, mientras mantuvo un Macro-F1 de training de 0.9267.

Conjunto	Macro-F1	Accuracy
Training	0.9267	0.9283
Validation	0.9341	0.9357

Generalization gap: -0.0074

Clase	Validation precision	Validation recall	Validation F1-score
Osmancik	0.9348	0.9541	0.9444
Cammeo	0.9369	0.9110	0.9238



En comparación con el modelo base, el ajuste redujo bastante la dependencia del modelo respecto a los datos de training, y de igual forma, la diferencia pequeña entre ambos conjuntos indica que la reducción de complejidad eliminó gran parte del overfitting observado en el modelo base sin deteriorar su capacidad de generalización.

VII. EVALUACIÓN FINAL Y COMPARACIÓN DE MODELOS

A. Evaluación final de la regresión logística

Después de completar el entrenamiento y diagnóstico mediante training y validation, la regresión logística se evaluó sobre el conjunto de prueba reservado. El modelo obtuvo un Macro-F1 de 0.9254 y una accuracy de 0.9265, resultados cercanos a los observados durante el desarrollo.

Conjunto	Macro-F1	Accuracy
Test	<u>0.9254</u>	<u>0.9265</u>

Clase	Test precision	Test recall	Test F1-score
Osmancik	0.9545	0.9151	0.9344
Cammeo	0.8924	0.9417	0.9164

En test, Osmancik obtuvo un F1-score de 0.9344 y Cammeo de 0.9164. La similitud con los resultados de training y validation confirma que el desempeño del modelo se mantuvo sobre observaciones que no participaron en el entrenamiento.

Regresión logística - Test

Osmancik	399	37
Cammeo	19	307

Clase real

Osmancik Cammeo

Clase predicha

B. Evaluación final del Random Forest Ajustado

Una vez seleccionada la configuración final utilizando únicamente training y validation, el Random Forest con max depth=2 se evaluó sobre el conjunto de prueba. El modelo obtuvo un Macro-F1 de 0.9241 y una accuracy de 0.9252. El Macro-F1 disminuyó 0.0100 respecto al obtenido en validation, pero permaneció cercano tanto a validation como a training, lo que indica que el desempeño observado durante el desarrollo se mantuvo sobre datos completamente reservados.

Conjunto	Macro-F1	Accuracy
Test	<u>0.9241</u>	<u>0.9252</u>

Clase	Test precision	Test recall	Test F1-score
Osmancik	0.9544	0.9128	0.9332
Cammeo	0.8899	0.9417	0.9151

En test se observó un comportamiento complementario entre las dos clases. Osmancik presentó mayor precisión, mientras que Cammeo obtuvo mayor recall. Esto indica que el modelo tendió a recuperar una mayor proporción de los granos Cammeo, a costa de clasificar algunos granos Osmancik como pertenecientes a esta variedad. A pesar de esta diferencia, los F1-scores de ambas clases se mantuvieron cercanos, con 0.93 para Osmancik y 0.92 para Cammeo.

Random Forest final - Test

Osmancik	398	38
Cammeo	19	307

Clase real

Osmancik Cammeo

Clase predicha

La matriz muestra 57 errores entre las 762 observaciones de test. La distribución de estos errores no evidencia un fallo concentrado exclusivamente en una de las variedades, aunque el modelo presentó mayor dificultad para distinguir algunos casos de Osmancik de la clase Cammeo.

C. Comparación final de los modelos

Modelo	Train Macro-F1	Validation Macro-F1	Test Macro-F1	Train-validation gap
Regresión logística	0.9250	0.9342	0.9254	-0.0092
Random forest final	0.9267	0.9341	0.9241	-0.0074

Ambos modelos alcanzaron un desempeño prácticamente equivalente. En validation sus Macro-F1 difirieron únicamente en 0.0001, mientras que en test la regresión logística obtuvo una ventaja de 0.0013. Las diferencias por clase también fueron mínimas, por lo que los resultados no muestran una ventaja predictiva relevante del Random Forest sobre la regresión logística.

Este resultado es especialmente relevante considerando que el Random Forest base presentaba un fuerte sobreajuste. El ajuste dirigido consiguió reducir su varianza y llevar su desempeño fuera de muestra al mismo nivel aproximado de la regresión logística; por lo tanto, la principal mejora obtenida mediante el ajuste no fue superar al otro algoritmo, sino transformar un modelo sobreajustado en uno con una capacidad de generalización estable.

D. Diagnóstico final de sesgo, varianza y ajuste

Modelo	Sesgo	Varianza	Nivel de ajuste
Regresión logística	Bajo	Baja	Adecuado
Random Forest base	Bajo	Alta	Overfitting
Random Forest final	Bajo	Baja	Adecuado

La regresión logística mantuvo resultados altos y próximos entre training, validation y test, por lo que presentó un comportamiento estable desde su configuración inicial. Random Forest, en cambio, comenzó con alta varianza debido a su ajuste perfecto sobre training y la caída posterior en validation. La restricción de profundidad corrigió este comportamiento: el modelo final obtuvo Macro-F1 similares en los tres conjuntos sin presentar la caída simultánea de desempeño que caracterizaría underfitting.

VIII. CONCLUSIONES

En este trabajo se implementaron y evaluaron dos modelos de clasificación para distinguir las variedades de arroz

Cammeo y Osmancik a partir de sus características físicas. La regresión logística manual alcanzó un Macro-F1 de 0.9254 en test, mientras que el Random Forest ajustado obtuvo 0.9241, por lo que ambos modelos presentaron una capacidad de clasificación casi equivalente sobre observaciones no utilizadas durante su desarrollo. Esto indica que, para este dataset, la mayor complejidad del Random Forest no produjo realmente una ventaja predictiva frente al modelo lineal.

La principal diferencia entre ambos modelos surgió durante su proceso de entrenamiento, pues la regresión logística mantuvo resultados similares entre training, validation y test, sin mostrar señales importantes de overfitting, y en contraste, el Random Forest base obtuvo un Macro-F1 perfecto de 1.0000 en training pero solamente 0.9245 en validation, evidenciando alta varianza. El ajuste de hiperparámetros demostró que al limitar la profundidad de los árboles el modelo se volvía más efectivo, pues con `max_depth=2`, el Macro-F1 pasó a 0.9267 en training y 0.9341 en validation, eliminando gran parte de la brecha de generalización sin provocar underfitting.

El análisis también demostró la importancia del preprocesamiento y de evaluar los modelos sin enfocarse solamente en accuracy. La eliminación de *Convex Area* también redujo redundancia respecto a *Area*, mientras que el uso de Macro-F1 permitió hacer una comparación equilibrada del desempeño sobre ambas clases. En la regresión logística, las altas correlaciones que permanecieron entre algunas variables ayudaron también a explicar la falta de significancia estadística individual de sus coeficientes, incluso si el modelo conservó un buen desempeño en las predicciones. En conclusión, los resultados demuestran que una evaluación separada de training, validation y test, acompañada de un diagnóstico de sesgo y varianza, fue necesaria para distinguir entre un modelo que simplemente se ajusta bien a los datos de entrenamiento y uno que genuinamente generaliza a nuevas observaciones, uno aplicable a la vida real.

IX. REFERENCIAS

“UCI Machine Learning Repository.”
<https://archive.ics.uci.edu/dataset/545/rice+cammeo+and+osmancik>

F. Lee, “Qué es la regresión logística,” Nov. 26, 2025.
<https://www.ibm.com/mx-es/think/topics/logistic-regression>

“RandomForestClassifier,” Scikit-learn. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

K. Sharma, “Mastering Random Forest Hyperparameter Tuning for Enhanced Machine Learning Models,” Medium, Nov. 10, 2024.
<https://medium.com/@kalpit.sharma/mastering-random-forest-hyperparameter-tuning-for-enhanced-machine-learning-models-2d1a8c6c426f>

V. Chugh, “F1 Score in Machine Learning: A Balanced Metric for Precision and Recall,” Datacamp.com, Nov. 12, 2025. <https://www.datacamp.com/tutorial/f1-score>

E. Kavlakoglu, “¿Qué es un random forest?,” Mar. 25, 2026. <https://www.ibm.com/mx-es/think/topics/random-forest>